

Introduction

Behavioral design patterns deal with the communication and responsibility between objects, helping to design more flexible and robust systems. One such pattern is the **Memento Pattern**, which provides a way to capture and restore an object's state without violating encapsulation. This is particularly useful in applications where undo or rollback functionality is required.

Imagine you're working in a document editing application. As you make changes to the text, you'd like the ability to undo your edits and revert to a previous version. Instead of exposing the internal structure of the document object, the system uses a memento to store its state at a given point in time. These mementos can later be used to restore the document to that state, all while keeping the implementation details hidden from the outside world.

In the following sections, we'll dive deeper into the **Memento Pattern**, understanding how it allows us to preserve snapshots of an object's state and restore them as needed, without breaching the principle of encapsulation.

Memento Pattern

Formal Definition

The **Memento Pattern** is a behavioral design pattern that allows an object to capture its internal state and restore it later without violating encapsulation. It is especially useful when implementing features like undo/redo or rollback.

Key Components

This pattern defines three key components:

- **Originator:** The object whose internal state we want to save and restore.
- **Memento:** A storage object that holds the snapshot of the originator's state.
- **Caretaker:** The object responsible for requesting the memento and keeping track of it. It neither modifies nor examines the contents of the memento.

Real-Life Analogy: Undo/Redo in Text Editors

Think of the Memento Pattern as an undo/redo mechanism. When you type or edit something in a text editor, the application captures snapshots of the document at different points. Each snapshot (memento) is stored by an external caretaker (like a history stack), and the editor (originator) can revert to these snapshots when needed, without exposing its internal logic.

A key strength of the pattern is that the originator alone is responsible for creating its snapshots, thus preserving encapsulation while still allowing state recovery.

Let's now understand the working of the **Memento Pattern** through the help of a problem statement.

Understanding the Problem

Assume we are building a **resume editor** where a user can make changes to their resume – such as name, education, experience, or skills, and may also want the ability to **undo or redo changes**. To do this, we need a way to

take a snapshot of the resume at any point in time and restore it later.

Below is a basic implementation trying to mimic this functionality:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
// Originator class: stores the current state of the resume
```

```
class ResumeEditor {
```

```
public:
```

```
    string name;
```

```
    string education;
```

```
    string experience;
```

```
    vector<string> skills;
```

```
};
```

```
// ResumeSnapshot acts like a memento, but isn't encapsulated properly
```

```
class ResumeSnapshot {
```

```
public:
```

```
    string name;
```

```
    string education;
```

```
string experience;
```

```
vector<string> skills;
```

```
// Constructor: captures the current state from ResumeEditor
```

```
ResumeSnapshot(const ResumeEditor& editor) {
```

```
    name = editor.name;
```

```
    education = editor.education;
```

```
    experience = editor.experience;
```

```
    skills = editor.skills; // Deep copy
```

```
}
```

```
// Restore function: applies the stored state back to ResumeEditor
```

```
void restore(ResumeEditor& editor) const {
```

```
    editor.name = name;
```

```
    editor.education = education;
```

```
    editor.experience = experience;
```

```
    editor.skills = skills; // Deep copy
```

```
}
```

```
};
```

```
// Main driver to demonstrate snapshot creation and restoration
```

```
int main() {

    ResumeEditor editor;

    editor.name = "Alice";

    editor.education = "B.Tech in CS";

    editor.experience = "2 years at ABC Corp";

    editor.skills = {"Java", "SQL"};


    // Step 1: Create a snapshot before making changes

    ResumeSnapshot snapshot(editor);


    // Step 2: Modify the resume

    editor.name = "Alice Johnson";

    editor.skills.push_back("Spring Boot");


    cout << "After changes:\n";

    cout << "Name: " << editor.name << endl;

    cout << "Skills: ";

    for (const auto& skill : editor.skills) cout << skill << " ";

    cout << endl;


    // Step 3: Restore previous state using snapshot
```

```
snapshot.restore(editor);
```

```
cout << "\nAfter undo:\n";
```

```
cout << "Name: " << editor.name << endl;
```

```
cout << "Skills: ";
```

```
for (const auto& skill : editor.skills) cout << skill << " ";
```

```
cout << endl;
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
// Originator class: stores the current state of the resume
```

```
class ResumeEditor {
```

```
    String name;
```

```
    String education;
```

```
    String experience;
```

```
    List<String> skills;
```

```
}
```

// ResumeSnapshot acts like a memento, but isn't encapsulated properly

```
class ResumeSnapshot {
```

```
    public String name;
```

```
    public String education;
```

```
    public String experience;
```

```
    public List<String> skills;
```

// Constructor: captures the current state from ResumeEditor

```
    public ResumeSnapshot(ResumeEditor editor) {
```

```
        this.name = editor.name;
```

```
        this.education = editor.education;
```

```
        this.experience = editor.experience;
```

```
        this.skills = new ArrayList<>(editor.skills); // Deep copy
```

```
    }
```

// Restore function: applies the stored state back to ResumeEditor

```
    public void restore(ResumeEditor editor) {
```

```
        editor.name = this.name;
```

```
        editor.education = this.education;
```

```
        editor.experience = this.experience;
```

```
        editor.skills = new ArrayList<>(this.skills); // Deep copy
```

```

    }
}

// Main driver to demonstrate snapshot creation and restoration
class Main {

    public static void main(String[] args) {

        ResumeEditor editor = new ResumeEditor();

        editor.name = "Alice";

        editor.education = "B.Tech in CS";

        editor.experience = "2 years at ABC Corp";

        editor.skills = new ArrayList<>(Arrays.asList("Java", "SQL"));

        // Step 1: Create a snapshot before making changes

        ResumeSnapshot snapshot = new ResumeSnapshot(editor);

        // Step 2: Modify the resume

        editor.name = "Alice Johnson";

        editor.skills.add("Spring Boot");

        System.out.println("After changes:");

        System.out.println("Name: " + editor.name);
    }
}

```



```

        System.out.println("Skills: " + editor.skills);

        // Step 3: Restore previous state using snapshot
        snapshot.restore(editor);

        System.out.println("\nAfter undo:");

        System.out.println("Name: " + editor.name);

        System.out.println("Skills: " + editor.skills);
    }
}

```

Originator class: stores the current state of the resume

```
class ResumeEditor:
```

```

    def __init__(self):

        self.name = ""

        self.education = ""

        self.experience = ""

        self.skills = []

```

ResumeSnapshot acts like a memento, but isn't encapsulated properly

```
class ResumeSnapshot:
```

```
def __init__(self, editor):

    # Constructor: captures the current state from ResumeEditor

    self.name = editor.name

    self.education = editor.education

    self.experience = editor.experience

    self.skills = editor.skills.copy() # Deep copy


def restore(self, editor):

    # Restore function: applies the stored state back to ResumeEditor

    editor.name = self.name

    editor.education = self.education

    editor.experience = self.experience

    editor.skills = self.skills.copy() # Deep copy


# Main driver to demonstrate snapshot creation and restoration

def main():

    editor = ResumeEditor()

    editor.name = "Alice"

    editor.education = "B.Tech in CS"

    editor.experience = "2 years at ABC Corp"

    editor.skills = ["Java", "SQL"]
```

Step 1: Create a snapshot before making changes

```
snapshot = ResumeSnapshot(editor)
```

Step 2: Modify the resume

```
editor.name = "Alice Johnson"
```

```
editor.skills.append("Spring Boot")
```

```
print("After changes:")
```

```
print("Name:", editor.name)
```

```
print("Skills:", editor.skills)
```

Step 3: Restore previous state using snapshot

```
snapshot.restore(editor)
```

```
print("\nAfter undo:")
```

```
print("Name:", editor.name)
```

```
print("Skills:", editor.skills)
```

```
if __name__ == "__main__":
```

```
    main()
```

Issues in the Above Code

- **No Caretaker Role**

The snapshot is being manually handled inside the `main()` method. There's no dedicated class to manage multiple states.

- **No Undo/Redo Stack**

Only a single snapshot is supported. You can't perform multiple levels of undo or redo.

- **Breaks Encapsulation**

The fields in `ResumeSnapshot` are public. This exposes internal details and violates encapsulation.

- **Tightly Coupled Implementation**

`ResumeSnapshot` directly accesses and depends on the internal structure of `ResumeEditor`. If the fields change, the snapshot class must change too.

- **No Abstraction**

There's no abstraction to hide how snapshots are created or restored. Everything is directly visible and modifiable.

The Solution

The issues in the previous implementation can be effectively solved using the **Memento Pattern**. This pattern enables the originator (the object whose state we want to save) to produce a memento (a snapshot of its internal state), which can then be managed by a caretaker. The key advantage is that the object's internal state is restored **without breaking encapsulation**, and we can maintain a **history of changes**.

The Memento Pattern introduces three components:

- **Originator:** The object whose state we want to capture and restore. (In this case: **ResumeEditor**)
- **Memento:** An immutable object that stores the internal state of the originator.
- **Caretaker:** The object that holds and manages multiple mementos, enabling undo operations. (In this case: **ResumeHistory**)

Here's the updated code implementing the **Memento Pattern**:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <stack>
```

```
using namespace std;
```

```
// Originator with Memento inside
```

```
class ResumeEditor {
```

```
private:
```

```
    string name;
```

```
    string education;
```

```
    string experience;
```

```
    vector<string> skills;
```

```
public:
```

```
void setName(const string& n) {  
    name = n;  
}
```

```
void setEducation(const string& e) {  
    education = e;  
}
```

```
void setExperience(const string& exp) {  
    experience = exp;  
}
```

```
void setSkills(const vector<string>& s) {  
    skills = s;  
}
```

```
void printResume() {  
    cout << "x:----- Resume -----" << endl;  
    cout << "Name: " << name << endl;  
    cout << "Education: " << education << endl;  
    cout << "Experience: " << experience << endl;
```

```
    cout << "Skills: ";

    for (const auto& skill : skills) cout << skill << " ";

    cout << endl;

    cout << "x:-----" << endl;

}
```

```
// Inner Memento class
```

```
class Memento {
```

```
private:
```

```
    string name;
```

```
    string education;
```

```
    string experience;
```

```
    vector<string> skills;
```

```
public:
```

```
    Memento(const string& n, const string& e, const string& exp, const  
vector<string>& s)
```

```
        : name(n), education(e), experience(exp), skills(s) {}
```

```
    friend class ResumeEditor;
```

```
};
```

```
// Save the current state as a Memento
```

```
Memento save() {
```

```
    return Memento(name, education, experience, skills);
```

```
}
```

```
// Restore state from Memento
```

```
void restore(const Memento& memento) {
```

```
    name = memento.name;
```

```
    education = memento.education;
```

```
    experience = memento.experience;
```

```
    skills = memento.skills;
```

```
}
```

```
};
```

```
// Caretaker
```

```
class ResumeHistory {
```

```
private:
```

```
    stack<ResumeEditor::Memento> history;
```

```
public:
```

```
    void save(ResumeEditor& editor) {
```



```
    history.push(editor.save());  
}
```

```
void undo(ResumeEditor& editor) {  
    if (!history.empty()) {  
        editor.restore(history.top());  
        history.pop();  
    }  
}  
};
```

```
// Main driver
```

```
int main() {  
    ResumeEditor editor;  
    ResumeHistory history;  
  
    editor.setName("Alice");  
    editor.setEducation("B.Tech CSE");  
    editor.setExperience("Fresher");  
    editor.setSkills({"Java", "DSA"});  
    history.save(editor);  
}
```

```
editor.setExperience("SDE Intern at TUF+");

editor.setSkills({"Java", "DSA", "LLD", "Spring Boot"});

history.save(editor);


editor.printResume(); // Shows updated experience

cout << endl;


history.undo(editor);

editor.printResume(); // Shows resume after one undo

cout << endl;


history.undo(editor);

editor.printResume(); // Shows resume after second undo (initial state)


return 0;

}


import java.util.*;


// Originator with Memento inside
```

```
class ResumeEditor {  
  
    private String name;  
  
    private String education;  
  
    private String experience;  
  
    private List<String> skills;  
  
  
    public void setName(String name) {  
  
        this.name = name;  
  
    }  
  
  
    public void setEducation(String education) {  
  
        this.education = education;  
  
    }  
  
  
    public void setExperience(String experience) {  
  
        this.experience = experience;  
  
    }  
  
  
    public void setSkills(List<String> skills) {  
  
        this.skills = skills;  
  
    }  
}
```

```
public void printResume() {  
  
    System.out.println("x:----- Resume -----");  
  
    System.out.println("Name: " + name);  
  
    System.out.println("Education: " + education);  
  
    System.out.println("Experience: " + experience);  
  
    System.out.println("Skills: " + skills);  
  
    System.out.println("x:-----");  
  
}
```

// Save the current state as a Memento

```
public Memento save() {  
  
    return new Memento(name, education, experience,  
List.copyOf(skills));  
  
}
```

// Restore state from Memento

```
public void restore(Memento memento) {  
  
    this.name = memento.getName();  
  
    this.education = memento.getEducation();  
  
    this.experience = memento.getExperience();  
  
    this.skills = memento.getSkills();  
  
}
```

```
}
```

```
// Inner Memento class
```

```
public static class Memento {
```

```
    private final String name;
```

```
    private final String education;
```

```
    private final String experience;
```

```
    private final List<String> skills;
```

```
    private Memento(String name, String education, String experience,  
List<String> skills) {
```

```
        this.name = name;
```

```
        this.education = education;
```

```
        this.experience = experience;
```

```
        this.skills = skills;
```

```
}
```

```
    private String getName() {
```

```
        return name;
```

```
}
```

```
    private String getEducation() {
```

```

        return education;
    }

    private String getExperience() {
        return experience;
    }

    private List<String> getSkills() {
        return skills;
    }
}

// Caretaker
class ResumeHistory {
    private Stack<ResumeEditor.Memento> history = new Stack<>();

    public void save(ResumeEditor editor) {
        history.push(editor.save());
    }
}

```

```
public void undo(ResumeEditor editor) {  
    if (!history.isEmpty()) {  
        editor.restore(history.pop());  
    }  
}  
}
```

// Main driver

```
public class Main {  
    public static void main(String[] args) {  
        ResumeEditor editor = new ResumeEditor();  
        ResumeHistory history = new ResumeHistory();  
  
        editor.setName("Alice");  
        editor.setEducation("B.Tech CSE");  
        editor.setExperience("Fresher");  
        editor.setSkills(Arrays.asList("Java", "DSA"));  
        history.save(editor);  
  
        editor.setExperience("SDE Intern at TUF+");  
        editor.setSkills(Arrays.asList("Java", "DSA", "LLD", "Spring Boot"));
```

```
history.save(editor);
```

```
editor.printResume(); // Shows updated experience
```

```
System.out.println("");
```

```
history.undo(editor);
```

```
editor.printResume(); // Shows resume after one undo
```

```
System.out.println("");
```

```
history.undo(editor);
```

```
editor.printResume(); // Shows resume after second undo (initial  
state)
```

```
}
```

```
}
```

Originator with Memento inside

```
class ResumeEditor:
```

```
def __init__(self):
```

```
    self._name = ""
```

```
    self._education = ""
```

```
    self._experience = ""
```

```
    self._skills = []
```



```
def setName(self, name):
```

```
    self._name = name
```

```
def setEducation(self, education):
```

```
    self._education = education
```

```
def setExperience(self, experience):
```

```
    self._experience = experience
```

```
def setSkills(self, skills):
```

```
    self._skills = skills
```

```
def printResume(self):
```

```
    print("x:----- Resume -----")
```

```
    print("Name:", self._name)
```

```
    print("Education:", self._education)
```

```
    print("Experience:", self._experience)
```

```
    print("Skills:", self._skills)
```

```
    print("x:-----")
```

```
# Save the current state as a Memento
```

```
def save(self):
```

```
    return self.Memento(self._name, self._education, self._experience,  
self._skills.copy())
```

```
# Restore state from Memento
```

```
def restore(self, memento):
```

```
    self._name = memento.getName()
```

```
    self._education = memento.getEducation()
```

```
    self._experience = memento.getExperience()
```

```
    self._skills = memento.getSkills()
```

```
# Inner Memento class
```

```
class Memento:
```

```
    def __init__(self, name, education, experience, skills):
```

```
        self.__name = name
```

```
        self.__education = education
```

```
        self.__experience = experience
```

```
        self.__skills = skills
```

```
    def getName(self):
```

```
        return self.__name
```

```
def getEducation(self):  
    return self.__education
```

```
def getExperience(self):  
    return self.__experience
```

```
def getSkills(self):  
    return self.__skills
```

Caretaker

```
class ResumeHistory:
```

```
    def __init__(self):  
        self.history = []
```

```
    def save(self, editor):  
        self.history.append(editor.save())
```

```
    def undo(self, editor):  
        if self.history:
```

```
editor.restore(self.history.pop())
```

```
# Main driver
```

```
def main():
```

```
    editor = ResumeEditor()
```

```
    history = ResumeHistory()
```

```
    editor.setName("Alice")
```

```
    editor.setEducation("B.Tech CSE")
```

```
    editor.setExperience("Fresher")
```

```
    editor.setSkills(["Java", "DSA"])
```

```
    history.save(editor)
```

```
    editor.setExperience("SDE Intern at TUF+")
```

```
    editor.setSkills(["Java", "DSA", "LLD", "Spring Boot"])
```

```
    history.save(editor)
```

```
    editor.printResume() # Shows updated experience
```

```
    print()
```

```
history.undo(editor)
```

```
editor.printResume() # Shows resume after one undo
```

```
print()
```

```
history.undo(editor)
```

```
editor.printResume() # Shows resume after second undo (initial state)
```

```
if __name__ == "__main__":
```

```
    main()
```

Let's now understand how the **Memento pattern** solves the previously discussed issues.

How Memento Pattern Solves The Issues

Issues	How Memento Pattern Fixes It
No Caretaker	ResumeHistory class manages all snapshots (mementos) and performs undo operations.
Only one level of undo	Stack<ResumeEditor.Memento> maintains history of states, enabling multiple undo levels.

Public fields in snapshot	Memento fields are private final , ensuring proper encapsulation.
Tight coupling with ResumeEditor	Memento acts as a data capsule, hiding internal structure of ResumeEditor .
Snapshot logic spread outside class	Snapshot creation/restoration is internal to ResumeEditor , improving cohesion.

Additionally, the Memento Pattern delegates the responsibility of creating state snapshots to the actual owner of the state, i.e., the originator itself. Since the originator has full access to its internal state, it is the most suitable component to generate accurate and complete mementos. This maintains encapsulation while still enabling full rollback capabilities.

When to Use Memento Pattern

The Memento Pattern is most useful in scenarios where an object's state needs to be saved and restored at various points in time, without exposing its internal structure. Consider using the Memento Pattern in the following situations:

- **You need to implement undo/redo functionality:**
The Memento Pattern allows you to store and restore previous states, enabling seamless undo/redo operations.
- **You want to preserve the encapsulation of the object's state:**
The pattern lets you save an object's internal state without exposing its private fields to the outside world.

- **You are handling non-trivial state history management:**
For scenarios requiring multiple checkpoints or rollbacks, mementos offer a structured and maintainable solution.
-

Advantages and Disadvantages of Memento Pattern

Pros

- **Preserves encapsulation**
The originator can save and restore its own state without exposing its internal structure.
 - **Simplifies undo/redo functionality**
By maintaining snapshots of state, the pattern provides a clean way to implement undo/redo features.
 - **Cleaner separation of concerns**
The originator handles state, while the caretaker manages history—leading to modular and maintainable code.
-

Cons

- **Can be memory-intensive if storing too many states**
Saving large or frequent snapshots can consume significant memory.
 - **Might introduce caretaker complexity**
The caretaker must manage memento creation, storage, and retrieval carefully, especially when there are many states.
 - **Needs careful management of old mementos**
Without proper pruning, the buildup of old mementos can lead to performance or memory issues.
-

Real Life Use Cases

The Memento Pattern is highly useful in systems where object states need to be saved and restored over time without exposing their internal structure. Here are two real-world examples of where this pattern can be applied effectively:

1. Text Editors (e.g., Notepad, Google Docs):

In text editors, users often rely on undo and redo functionalities to reverse or repeat changes. Every time the user makes an edit, the current state of the document can be stored as a memento. When the user presses undo, the editor restores the previous state from the most recent memento. This allows users to seamlessly navigate back and forth through changes without accessing or modifying the internal details of the document object.

2. Graphic Design or Drawing Applications:

Applications like Photoshop or Figma allow users to apply changes step by step (e.g., drawing, coloring, transforming objects). With each significant operation, a snapshot (memento) of the canvas or component's state is saved. Users can then use undo to revert to a specific state. This keeps the design process non-destructive and flexible while ensuring encapsulation of the canvas data.

These examples demonstrate how the Memento Pattern enables powerful undo/redo support and history management, all while preserving encapsulation and reducing system complexity.

Class Diagram

